

Section: CS

Understanding Prevalence of Malicious Advertising in Search Engine Results

Chris Humphreys

Problem Statement:

Malicious actors are abusing advertising infrastructure to deliver malware and related malicious content like ClickFix. While the problem is known to exist due to the large number of individuals who have fallen victim to these scams, there has been very little attempt to understand just how prevalent these malicious advertisements are in search engine results in the last roughly ten year time period. Prior to this time period, there were a number of documented attempts to understand the prevalence of these ads, but I have not been able to find any recent research on this topic. The problem this research will address is the problem of not having a clear picture of how prevalent malvertising currently is.

This is a significant alteration from my original draft proposal based on feedback from Dr. Ahamad. Two of his recommendations were that I review the literature for techniques relevant to the problem I wanted to address, and to plan on how I would detect if a search result was safe or malicious. In conducting the literature review to address the first question, it quickly became apparent that while this was a somewhat prominent area of study in the past, not much work has been published on this in the last decade. This makes it difficult to address question 2, as there is very little current research on how to detect if a search result is malicious or not.

So now, the overall problem I submitted in my original proposal, that being the problem of malvertising in general, has now been much more tightly scoped to address the lack of research on of the prevalence of the malicious advertisements since roughly 2016.

Solution Statement:

This research will attempt to measure the prevalence of malicious advertisements in search engine results in order to better understand both how adversaries are hiding malicious advertisements, and to try to understand how prevalent these ads are in search engine results. This is being done in order to address an area of study that has been neglected for some time. Additionally, this work can be built upon to create even better measures of the problem, or it can be used as a way to measure how effective various attempts to block or fix the malvertising problem are. It will also give us a measurement we can use against the older work to see if the problem of malvertising has increased or decreased since that work was undertaken.

Completed Tasks (Last 2 Weeks):

- Chose OxyLabs as my SERP provider. I looked at a huge number and went with OxyLabs as they provide a residential proxy option as well which I may be able to use as part of my Detector code. Almost went with SerpApi, but they were more expensive.
- I looked at a bunch of residential proxies as well and at this time am just planning on going with OxyLabs as they offer both services. If OxyLabs is not getting the job done, or if I find a really good alternative I may switch, but for not OxyLabs is the plan.
- As far as alternatives for wepawet, while I don't have an official decision made yet, I have identified a number of good options. Since I work at CrowdStrike I will likely go with Hybrid-analysis as I bet I can use it for free. Other options are JoseSandbox and UrlScan.io.

- Got my dev environment set up, including virtual python environment for dev work, github repo for storing code, and getting emacs plugins and things that I was missing.
- After setting up my OxyLabs account read thru docs and learned how the API calls work
- Collector code (implemented as a python class) is complete and working as I wanted. Code can be used to get a search engine result on demand and can be used to pull as many results as desired.
- Got collector working as a multi-threaded application. Adding a completed task for this due to the fact that I am extremely proud I got this working. I am allowed to run up to 10 requests at once by the API provider, so I wanted my code to take advantage of this. I have not programmed with multiple threads other than in little class projects years ago, so this took a huge amount of time and was very frustrating to get going. At the end of the day though it is working!
- Researched a bunch of Domain blacklists and found some really good options. Two in particular stand out, the first is a github repo just called malicious-domains, that pulls in domains from a number of open source feeds once an hour. The other was one called tweetfeed.live which adds domains found in tweets from known security researchers. Ideally, I will combine both and use both sources to compare domains agains.

Tasks for the Next Project Report:

Next project report is really all about the Detector, and my biggest concern is that its gonna take longer than I set aside. I am pretty sure this one is gonna need to be multi-threaded as well as so much is gonna depend on waiting for network requests to resolve. I also will need to program a way to have it compare identified domains against my domain blacklist as the first oracle to consult in determining maliciousness. While this seems like only a few items, each of these are going to be a huge amount of work.

Questions I have or Issues I'm running into:

So far so good, not a ton of questions or issues. One big concern I have is if I am using sites like joesandbox or hybrid-analysis as Oracles for determination of site maliciousness, is how them calling the site will affect the analysis. For example, I wonder if sites check to see if it is Joe Sandbox doing the calling, and if so, do they send a redirect to a benign site instead of what is normally there, throwing off my analysis.

Methodology Paragraph Summary:

My proposed methodology is to use a (Search Engine Results Page) API provider to collect advertisement URLs from Google. After collecting the advertisement URLs, I will use a residential proxy service in conjunction with a "collector" and "detector" program which I will write to collect the contents of the sites linked to from the ads. Then I will use oracles based on the ones used in the 2014 paper "The Dark Alleys of Madison Avenue: Understanding Malicious Advertisements"[18] to determine if the site is malicious or not. The results and number of malicious ads identified can be compared with the results of previous studies to better understand the prevalence of modern malvertising.

This methodology was based on Dr. Ahamad's feedback, the first piece of which was that I need to provide some details regarding the feasibility and scope of the project. In terms of feasibility, the biggest concern I have is that of rate limiting and blocking on Google's side. Google will likely attempt to block automated attempts to scan their search results for ads. Rather than make the requests directly, I will use the services of a third party SERP provider which provides a proxy API I can use to make the requests on my behalf and bypass this issue.

The second concern I have is that many of these malicious ads may use “cloaking” in which if they detect a scraper or researcher they send the user to a benign site rather than the malicious one. In the 2015 paper “Automated Collection and Analysis of Malware Disseminated via Online Advertising”[16] a model is suggested for a “Detector” of malicious advertisements that opens pages to determine if the URL leads to a file download. I plan on borrowing this model to create a similar detector which I will run over the results of my search engine scrape. In addition to looking for file downloads, I will also look for strings indicative of ClickFix or other malicious content. Additionally, I will use some sort of residential proxy for these requests to make it more difficult for malicious advertisements to realize I am a researcher and send me to an otherwise benign site.

The scope of work now, collecting the ads from the SERP provider and then checking for malicious items via the Detector I will create is feasible for the time frame we have. Ideally, this will all result in a set of data that can be used to compare against the previous papers to determine if the problem of malvertising is increasing or decreasing, and provide a modern baseline of malicious advertisement levels for future research to build upon.

Timeline:

Week #	Description of Task	Status
W1 (Aug 25 to Aug 31)	Brainstorm Ideas for Project → Firefox Add On	Completed
W1	Research tutorials and lessons on building Firefox AddOns	Complete
W1	Build borderify sample AddOn from Firefox documentation	Complete
W1	Build Beastify AddOn from Firefox documentation	Complete
W2 (Aug 31 to Sept 6)	Brainstorm and determine best course of action based on initial feedback from Dr. Ahmad	Complete
W2	Search ACM articles for anything related to malvertising, read pertinent articles and updated proposal based on findings.	Complete
W2	Search IEEE for any articles related to malvertising, read pertinent articles and update proposal based on findings.	Complete
W2	Do a web search engine based search for non-academic research and findings regarding malvertising.	Complete
W2	Synthesize all findings from literature review and then update proposal and problem statement based on findings.	Complete
W2	Review Dr. Ahmad’s feedback again in relation to updated proposal and problem statement ideas. Update to include all feedback.	Complete
W3 (Sept 7 to Sept 14)	Determine which SERP is best suited for my task, set up account and test functionality	Complete
W3	Determine which residential proxy is best suited for my task, set up account and test functionality	Complete
W3	Find a modern working alternative for wepawet oracle. Set up account and test functionality	Complete
W3	Make skeleton of “collector” code that will	Complete

	utilize the SERP API to pull in search engine results.	
W4 (Sept 14 to Sept 20)	Complete code for “Collector” and have it fully functional and ready to be used.	Complete
W4	Find one or more “Domain blacklist” sites I can use to compare ads against to determine if malicious.	Complete
W5 (Sept 21 to Sept 27)	Make skeleton of Detector code.	Incomplete
W5	Integrate residential proxy with detector code so that requests are less likely to be identified by adversaries	Incomplete
W6 (Sept 28 to Oct 4)	Complete Detector code and ensure it has the ability to read from the Ad URL’s as expected	Incomplete
W7 (Oct 5 to Oct 11)	Combine detector and collector code into a single application	Incomplete
W7	Add capability that allows findings to be compared against the “blacklisted Domain Oracle”	Incomplete
W8 (Oct 12 to Oct 18)	Add capability that allows findings to be compared against the “VirusTotal” malware hash Oracle.	Incomplete
W8	Add capability that allows findings to be compared against the “dynamic site analysis” dashboard.	Incomplete
W9 (Oct 19 to Oct 25)	Combine all Oracle and previous code into one cohesive application that can be used to start scanning.	Incomplete
W9	Do at least 1 successful test run of 1000 pages of search engine results,	Incomplete
W10 (Oct 26 to Nov 1)	Based on results from W9 test run, update code to solve any identified issues	Incomplete
W11 (Nov 2 to Nov 8)	Based on test run from W9, determine ideal number of runs to use.	Incomplete
W11	Run a scan based on number of times decided on above.	Incomplete
W11	Vet results of full run looking for code errors, bugs etc. Fix any identified issues	Incomplete
W12 (Nov 9 to Nov 15)	Do true scan, the results of which will be the final results presented in the paper.	Incomplete
W13 (Nov 16 to Nov 22)	Analyze findings, determine conclusions, compare against results from earlier studies	Incomplete
W13	Complete first draft of Final Paper based on results.	Incomplete
W14 (Nov 23 to Nov 29)	Thanksgiving	
W15 (Nov 30 to Dec 6)	Write final draft of paper	Incomplete
W15	Submit!	Incomplete

Evaluation:

[Include any evaluation plans and/or results by Progress Report 4. This may expand as you finalize the report.]

Report Outline:

[Include an outline of your final report by Progress Report 5. This may expand as you finalize the report.]

References:

- [1] anon. Malvertising What is malvertising and how to protect yourself. Malwarebytes. Retrieved September 6, 2026 from <https://www.malwarebytes.com/malvertising>
- [2] anon. 2026. Malvertising & Ad Quality Index - Risky Business. Retrieved September 6, 2026 from https://www.confiant.com/hubfs/Team%20-%20Marketing/THE%20MAQ%20INDEX%202026_%20RISKY%20BUSINESS.pdf
- [3] Contributors to Wikimedia projects. 2010. Malvertising. Wikipedia. Retrieved September 6, 2026 from <https://en.wikipedia.org/wiki/Malvertising>
- [4] Gadget Hacks. 2026. Ad Blockers That Still Work in Chrome: 4 Ways to Block Ads. Gadget Hacks. Retrieved September 6, 2026 from <https://android.gadgethacks.com/how-to/ad-blockers-that-still-work-in-chrome-4-ways-to-block-ads/>
- [5] David Kasabji. 2025. From SEO Poisoning to Malware Deployment: Malvertising campaign uncovered. Conscia. Retrieved September 5, 2026 from <https://conscia.com/blog/from-seo-poisoning-to-malware-deployment-malvertising-campaign-uncovered/>
- [6] Jérôme Segura. 2016. A Look Behind The Skype Malvertising Campaign | Malwarebytes Labs. Malwarebytes. Retrieved September 5, 2026 from <https://www.malwarebytes.com/blog/news/2016/02/a-look-behind-the-skype-malvertising-campaign>
- [7] Dhanur Sehgal. 2026. What Is Malvertising? Examples, Risks & Prevention. The Blockify Blog. Retrieved September 5, 2026 from <https://getblockify.com/blog/malvertising/>
- [8] Michael Steele. 2026. SourTrade: Browser-Assembled Malware Delivered Through Malvertising. Confiant. Retrieved September 5, 2026 from <https://blog.confiant.com/p/sourtrade-browser-assembled-malware>
- [9] George Thomas. 2015. Malvertising up 325% - Are the Adblockers Working?. Retrieved September 6, 2026 from <https://web.archive.org/web/20160130161632/http://blog.check-and-secure.com/091015-malvertising-up-325-are-adblockers-working/>
- [10] Shubham Kumar, Siddharth Swarup Rautaray, and Manjusha Pandey. 2017. Malvertising: A case study based on analysis of possible solutions. In 2017 International Conference on Inventive Computing and Informatics (ICICI), November 2017. IEEE, 288–291. <https://doi.org/10.1109/icici.2017.8365356>
- [11] Joshua Rolon, Cheryl Hinds, and Felicia Doswell. 2019. SpartanShield: A Layered Defense against Malvertising. In 2019 International Conference on Computational Science and Computational Intelligence (CSCI), December 2019. IEEE, 185–190. <https://doi.org/10.1109/csci49370.2019.00038>
- [12] Eric Blancaflor, Jon Eril R. Aparize, Joyous M. Geronimo, Ethan Joseph S. Gonzales, and Joyful L. Morilla. 2024. Malvertising in AI-Powered Chat Platforms: Utilizing Integrated Artificial Intelligence Solutions for Threat Detection and Mitigation. In 2024 8th International Conference on Communications and Future Internet (ICCFI), July 19, 2024. IEEE, 67–74. <https://doi.org/10.1109/iccfi63302.2024.00018>

- [13] Ankita Kumari and Ishu Sharma. 2024. Mitigating Malvertising Threats: An Exploration of Machine Learning Classification Algorithms for Effective Detection. In 2024 International Conference on Advancements in Smart, Secure and Intelligent Computing (ASSIC), January 27, 2024. IEEE, 1–6. <https://doi.org/10.1109/assic60049.2024.10508033>
- [14] Chin-Tser Huang, Muhammad N. Sakib, Charles Kamhoua, Kevin Kwiat, and Laurent Njilla. 2017. A game theoretic approach for inspecting web-based malvertising. In 2017 IEEE International Conference on Communications (ICC), May 2017. IEEE, 1–6. <https://doi.org/10.1109/icc.2017.7996807>
- [15] Chin-Tser Huang, Muhammad N. Sakib, Charles A. Kamhoua, Kevin A. Kwiat, and Laurent Njilla. 2020. A Bayesian Game Theoretic Approach for Inspecting Web-Based Malvertising. IEEE Trans. Dependable and Secure Comput. 17, 6 (November 2020), 1257–1268. <https://doi.org/10.1109/tdsc.2018.2866821>
- [16] Muhammad N. Sakib and Chin-Tser Huang. 2015. Automated Collection and Analysis of Malware Disseminated via Online Advertising. In 2015 IEEE Trustcom/BigDataSE/ISPA, August 2015. IEEE, 1411–1416. <https://doi.org/10.1109/trustcom.2015.539>
- [17] Tiliang Zhang, Hua Zhang, and Fei Gao. 2013. A Malicious Advertising Detection Scheme Based on the Depth of URL Strategy. In 2013 Sixth International Symposium on Computational Intelligence and Design, October 2013. IEEE, 57–60. <https://doi.org/10.1109/iscid.2013.128>
- [18] Apostolis Zarras, Alexandros Kapravelos, Gianluca Stringhini, Thorsten Holz, Christopher Kruegel, and Giovanni Vigna. 2014. The Dark Alleys of Madison Avenue: Understanding Malicious Advertisements. In Proceedings of the 2014 Conference on Internet Measurement Conference, November 05, 2014. ACM, 373–380. <https://doi.org/10.1145/2663716.2663719>
- [19] Xinyu Xing, Wei Meng, Byoungyoung Lee, Udi Weinsberg, Anmol Sheth, Roberto Perdisci, and Wenke Lee. 2015. Understanding Malvertising Through Ad-Injecting Browser Extensions. In Proceedings of the 24th International Conference on World Wide Web, May 18, 2015. International World Wide Web Conferences Steering Committee, 1286–1295. <https://doi.org/10.1145/2736277.2741630>
- [20] Zhou Li, Kehuan Zhang, Yinglian Xie, Fang Yu, and XiaoFeng Wang. 2012. Knowing your enemy. In Proceedings of the 2012 ACM conference on Computer and communications security, October 16, 2012. ACM, 674–686. <https://doi.org/10.1145/2382196.2382267>

Appendix

If there are notes, sources, figures you want to reference, or your draft writing please include that here.

[The inclusion of draft writing for the final report should begin being included by Progress Report 3.]